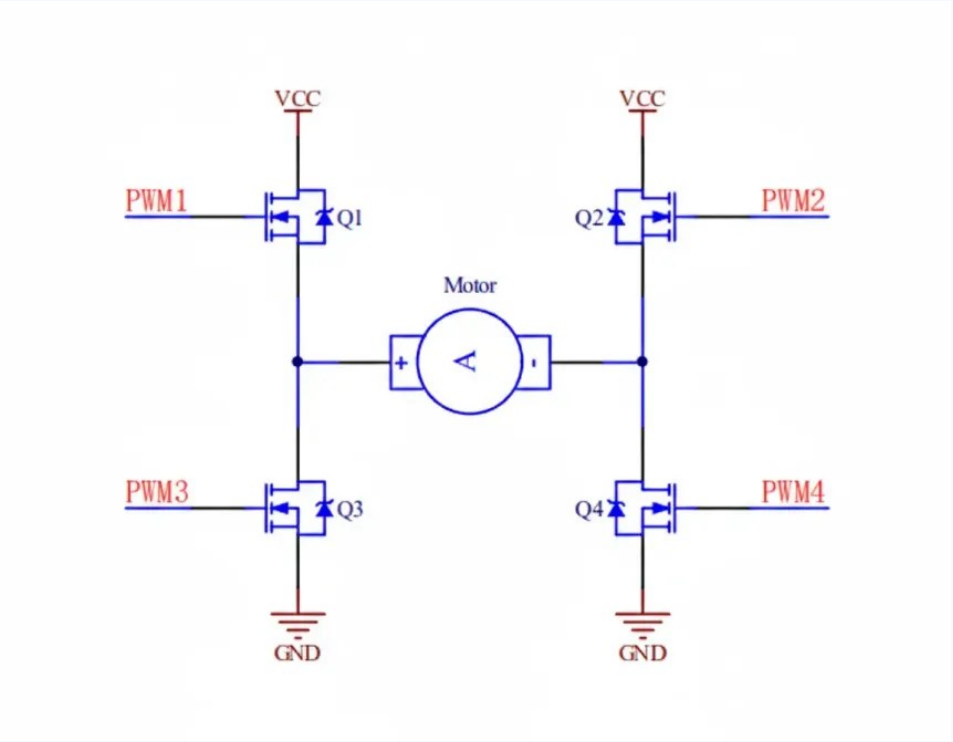
DC Motor Drive and Open-Loop Control

Dual H-Bridge Mechanics and Chassis Motion

Session 04

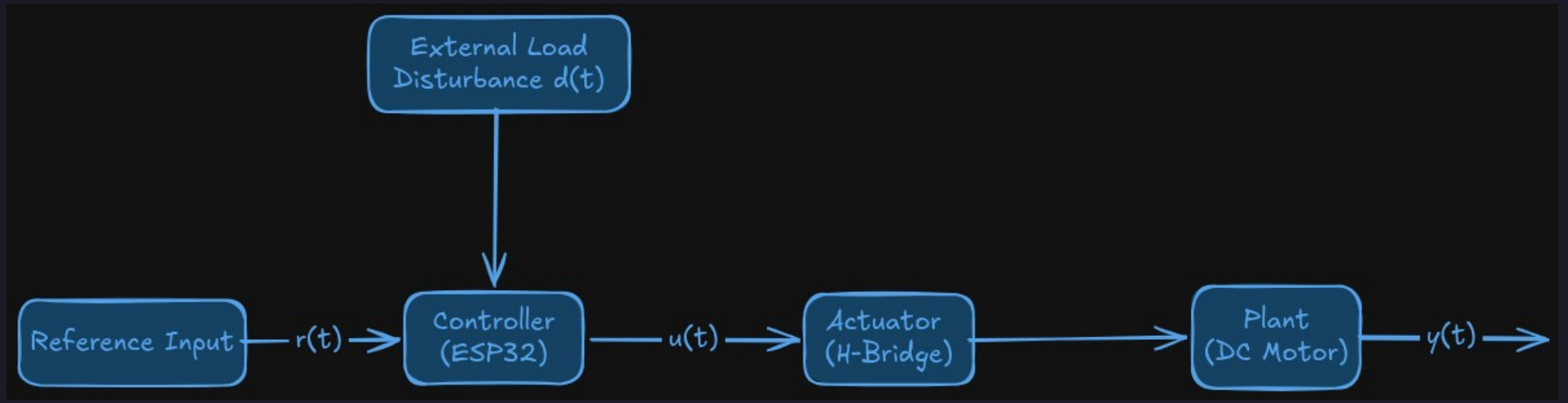


Contents

01	**Open-Loop Control**	Commands, actuators, plants, and disturbances
02	**DC Motors**	Voltage balance, torque, and inductive loads
03	**H-Bridge Switching**	Direction, braking, current decay, and dead time
04	**Implementation**	Wire, drive, measure, and debug the chassis

Source: course PDF, pp. 1–17

From a Command to Physical Motion



The physical load disturbance acts on the motor and chassis.

Source: course PDF, pp. 1–2

The Four Functional Blocks

Block	Role
Reference input	Requested speed or motion from the user or planner.
Controller	ESP32 firmware maps the reference to a drive command.
Actuator	H-bridge switches electrical power into the motors.
Plant	Motors, gearboxes, wheels, and chassis produce motion.

A commanded PWM duty is not a measurement of wheel speed.

Source: course PDF, pp. 1–2

Open-Loop Control: No Output Feedback

$$u(t) = f[r(t)]$$

$$D = \frac{|N_{\mathrm{PWM}}|}{255}, \qquad 0 \le D \le 1$$

$$V_{\mathrm{avg}} \approx D\, V_M$$

The voltage approximation assumes ideal drive/brake PWM and ignores driver losses.

Source: course PDF, pp. 2

Open Loop and Closed Loop

Open loop	The command determines the drive output. There is no measured speed correction.
Closed loop	Measure actual motion and compare it with the desired reference.
Trade-off	Open loop is simple; feedback can correct load-dependent speed errors.

This session establishes the motor-drive layer before adding motion feedback.

Source: course PDF, pp. 2

A Disturbance Changes the Actual Speed

$$e(t) = r(t) - y(t)$$

$$D_1 = D_2, \qquad \omega_1 \neq \omega_2 \text{ is possible}$$

Friction, battery sag, and unequal wheel loading can change motion at fixed duty.

Source: course PDF, pp. 1–2

Keep Logic and Motor Power Separate

ESP32 GPIO	Supplies control signals at 3.3 V logic; it is not a motor-power output.
Motor supply	Provides the armature current through the H-bridge and heavy-current return path.
Two failure paths	Excess current can damage GPIO outputs; inductive switching can produce voltage spikes.

Connect motors to the driver outputs, with a shared logic ground.

Source: course PDF, pp. 2–3

The Armature Voltage Balance

$$V_{\text{app}}(t) = k_e \omega(t) + I(t) R_a + L_a \frac{dI(t)}{dt}$$

Applied voltage balances back EMF, winding resistance, and inductive voltage.

Source: course PDF, pp. 3

Read the Motor Model

Symbol	Meaning	Unit
Vapp	Applied terminal voltage	V
ω	Motor-shaft angular speed	rad/s
I	Armature current	A
Ra / La	Winding resistance / inductance	Ω / H
ke / kt	Back-EMF / torque constant	V·s/rad / N·m/A

Keep shaft speed and gearbox output speed distinct when using motor constants.

Source: course PDF, pp. 3

Steady-State Voltage and Torque

$$\frac{dI}{dt} = 0 \quad \Rightarrow \quad V_{\text{app}} = k_e \omega + I R_a$$

$$\tau = k_t I \quad \Rightarrow \quad I = \frac{\tau}{k_t}$$

Torque is proportional to current; back EMF is proportional to shaft speed.

Source: course PDF, pp. 3

Derive the Speed-Torque Relationship

$$V_{\text{app}} = k_e \omega + \frac{R_a}{k_t} \tau$$

$$\omega = \frac{V_{\text{app}}}{k_e} - \frac{R_a}{k_e k_t} \tau$$

$$k_e = k_t \text{ in consistent SI units} \quad \Rightarrow \quad \omega = \frac{V_{\text{app}}}{k_t} - \frac{R_a}{k_t^2} \tau$$

The slope is –Ra/(ke kt); the squared term applies to kt, not to Ra.

Source: course PDF, pp. 3

No-Load and Stall Limits

$$\omega_0 \approx \frac{V_{\text{app}}}{k_e} \qquad (\tau \approx 0)$$

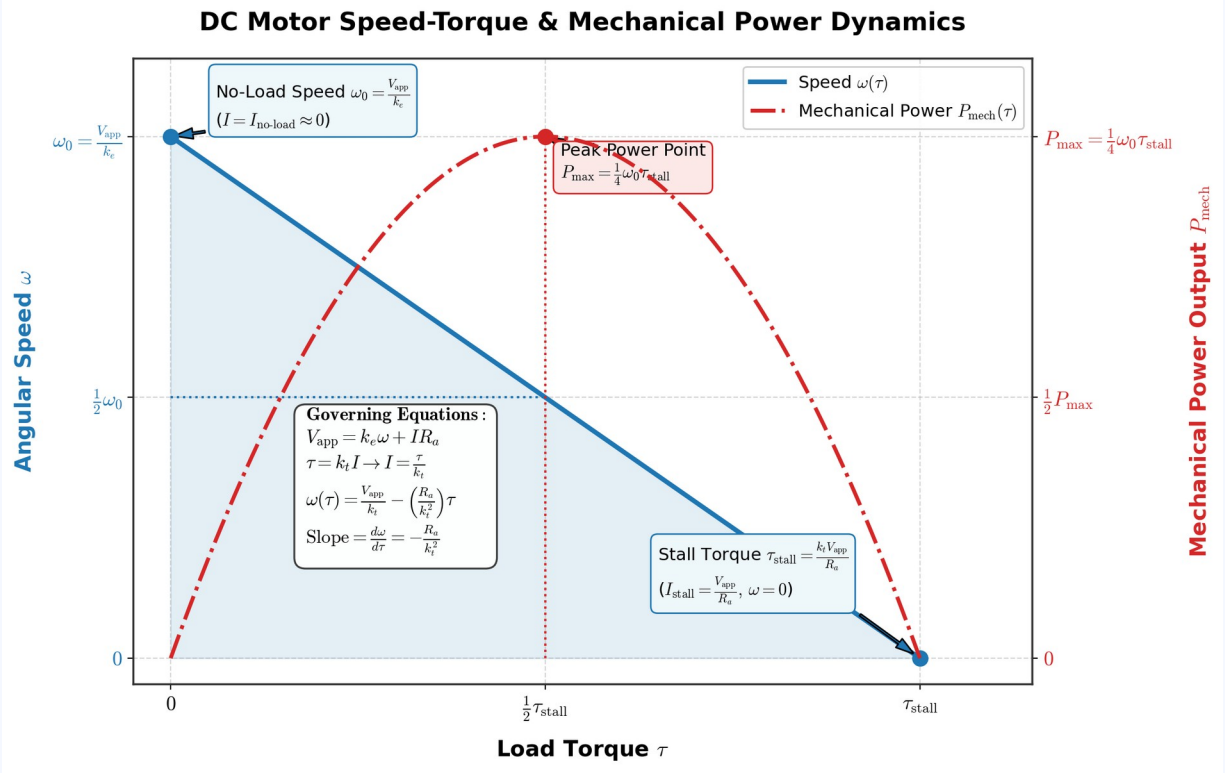
$$I_{\text{stall}} = \frac{V_{\text{app}}}{R_a} \qquad (\omega = 0)$$

$$\tau_{\text{stall}} = k_t I_{\text{stall}}$$

Stall is a high-current condition; real no-load current also supplies motor losses.

Source: course PDF, pp. 3–4

Speed, Torque, and Mechanical Power



Original course figure: ideal speed falls as load torque increases.

Source: course PDF, pp. 4

Mechanical Power Along the Curve

$$P_{\text{mech}} = \tau\omega$$

$$\tau = \frac{\tau_{\text{stall}}}{2}, \quad \omega = \frac{\omega_0}{2}$$

$$P_{\text{max}} = \frac{1}{4}\tau_{\text{stall}}\omega_0$$

This ideal peak-power point is not a continuous operating recommendation.

Source: course PDF, pp. 4

Parallel Motors Add Channel Current

$$I_{\text{channel}} = I_{\text{front}} + I_{\text{rear}}$$

$$2 \times (0.25 \text{ to } 0.40)\,\text{A} = 0.50 \text{ to } 0.80\,\text{A}$$

$$I_{\text{four motors}} = 1.00 \text{ to } 1.60\,\text{A}$$

Arithmetic uses the PDF’s per-motor example; measure the actual motor demand.

Source: course PDF, pp. 2, 8

Inductive Energy and Switching Voltage

$$E_L = \frac{1}{2} L_a I^2$$

$$v_L = L_a \frac{dI}{dt}$$

Current cannot change instantaneously: switching must provide a recirculation path.

Source: course PDF, pp. 4–5

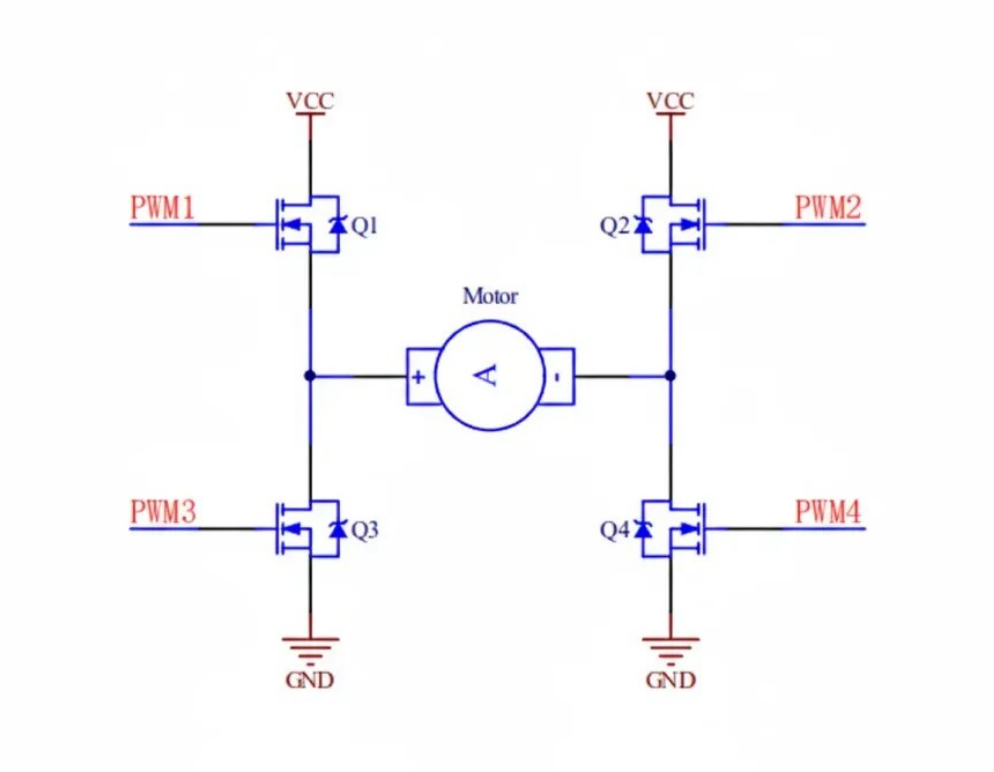
Flyback and Current Recirculation

Switch turns off	Stored magnetic energy keeps armature current flowing in its original direction.
Current path	Body diodes or driven MOSFETs provide a path while the winding current decays.
Supply behavior	Some paths return energy to the supply; rail voltage and decoupling still matter.

The recirculation path depends on the driver state and current direction.

Source: course PDF, pp. 4–5

The Four-Switch H-Bridge

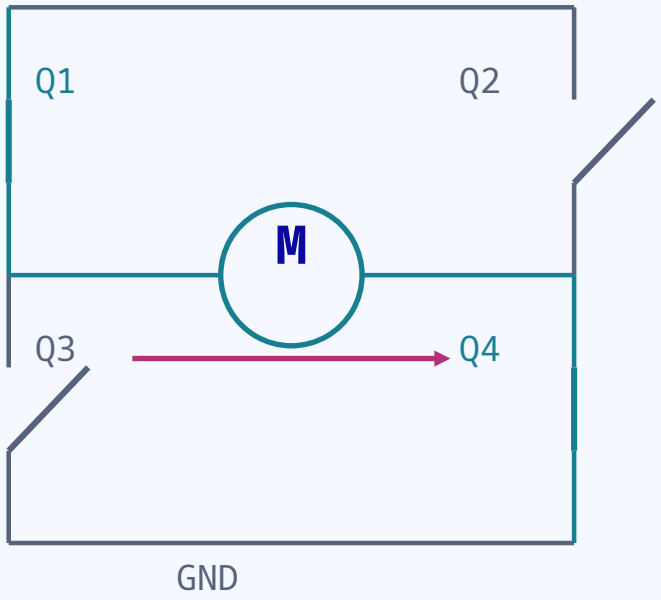


Use the picture numbering: Q1/Q2 are high-side; Q3/Q4 are low-side.

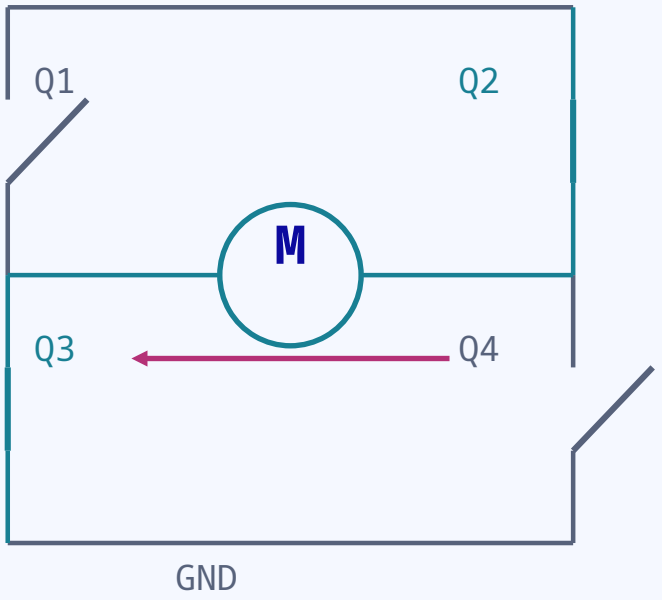
Source: course PDF, pp. 5–6

Forward and Reverse Current Paths

FORWARD / Q1 + Q4 (VM)



REVERSE / Q2 + Q3 (VM)



Select one diagonal pair; motor wiring determines the physical rotation direction.

Source: course PDF, pp. 6

Ideal Switch States

Mode	Q1	Q2	Q3	Q4	Motor state
Forward	ON	OFF	OFF	ON	+VM
Reverse	OFF	ON	ON	OFF	–VM
Low-side brake	OFF	OFF	ON	ON	Terminals shorted
Coast command	OFF	OFF	OFF	OFF	Outputs high-Z

These are transistor states, not the integrated driver’s input truth table.

Source: course PDF, pp. 6–7

Braking and Coasting Are Different

Dynamic braking	Both motor terminals are tied together. Generated current opposes the ongoing motion.
Coasting	Outputs are high impedance; the rotor slows through mechanical and electrical losses.
During PWM	Current decay describes the winding current during the inactive part of each PWM cycle.

Do not use “brake/coast” and “fast/slow decay” as interchangeable names.

Source: course PDF, pp. 6–7

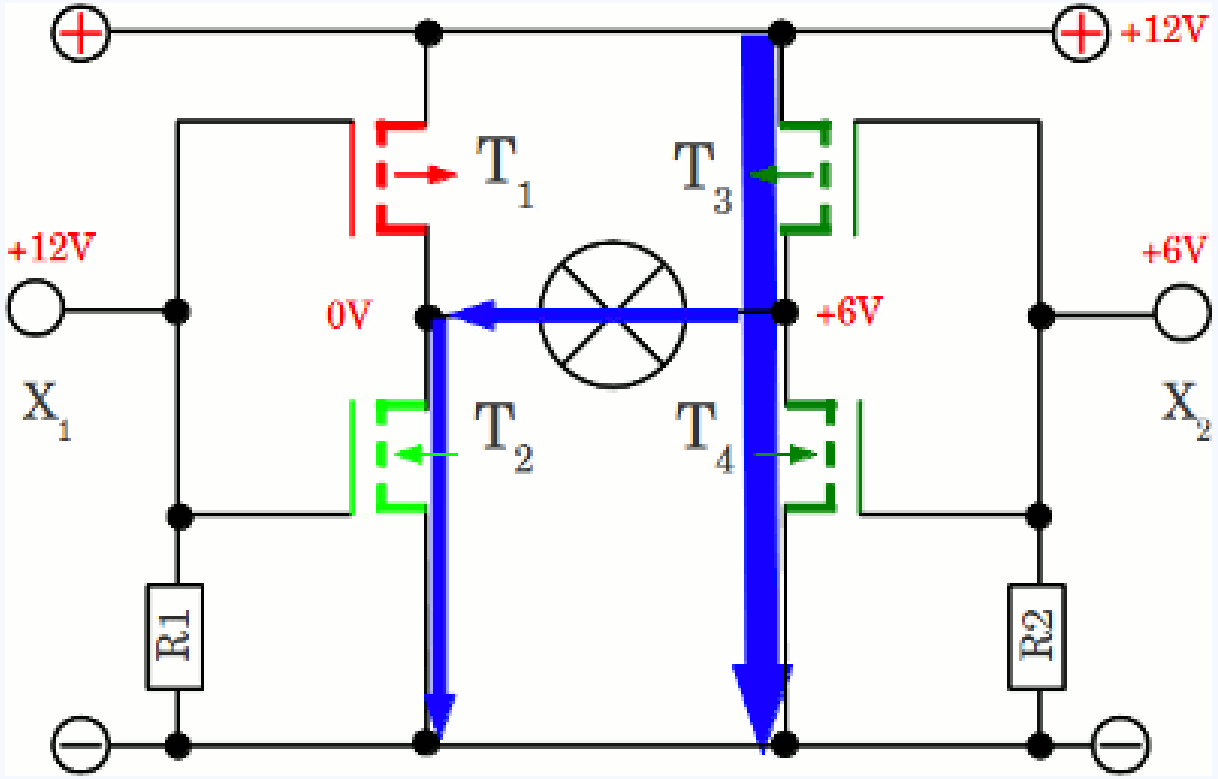
Slow and Fast Current Decay

Slow decay	Shorted-terminal recirculation usually applies a relatively small opposing winding voltage.
Fast decay	A path through the opposite diagonal or diodes applies a larger opposing voltage.
Practical effect	The off-state changes current ripple, response speed, and the duty-to-speed relation.

Include back EMF and resistance when analyzing a rotating motor.

Source: course PDF, pp. 7

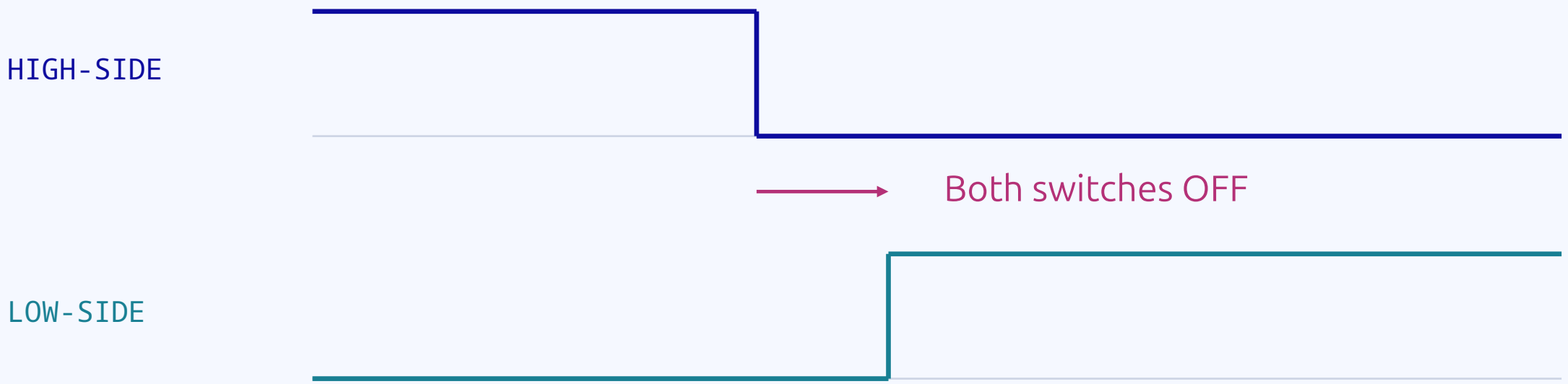
Shoot-Through: A Supply Short Circuit



Never turn on the high-side and low-side switches of the same leg together.

Source: course PDF, pp. 7

Dead Time Between Complementary Switches



The integrated driver inserts dead time; motor reversal is a separate current transient.

Source: course PDF, pp. 8

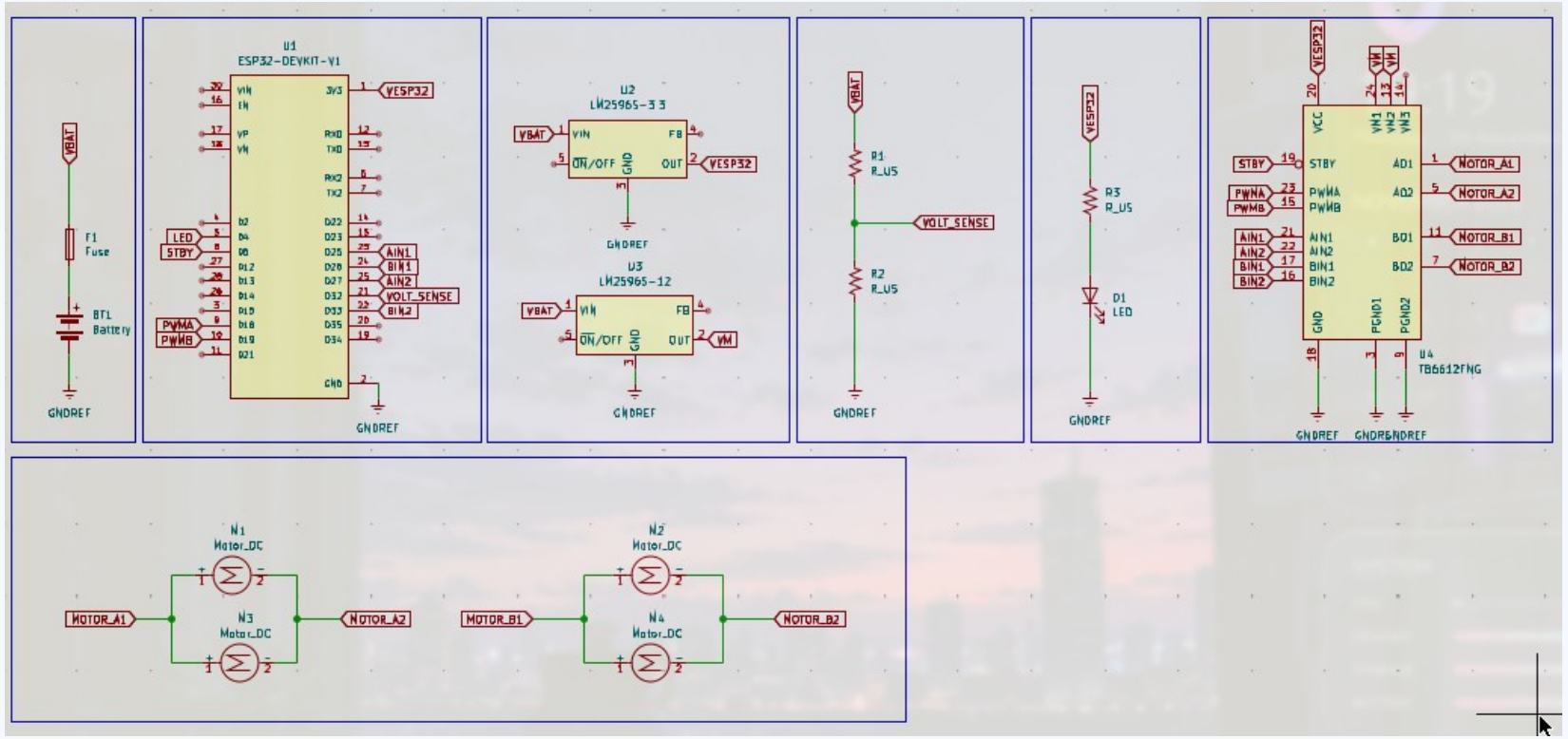
TB6612FNG: Command the Actual IC

STBY	IN1	IN2	PWM	Output mode
LOW	X	X	X	Standby / high-Z
HIGH	HIGH	LOW	HIGH	Forward polarity
HIGH	LOW	HIGH	HIGH	Reverse polarity
HIGH	HIGH	HIGH	X	Short brake
HIGH	LOW	LOW	HIGH	Stop / high-Z
HIGH	HIGH	LOW	LOW	Short brake (PWM off)

X = either level. Use the actual driver table when choosing stop and PWM behavior.

Source: course PDF, pp. 6–8, 11–12

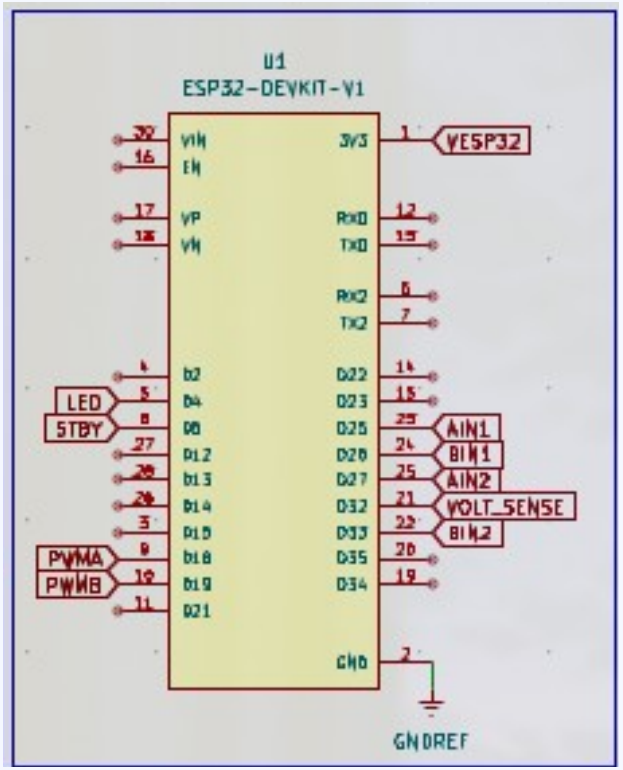
The Complete Motor-Drive Schematic



Original drawing: follow net labels between ESP32, driver, and the two motor pairs.

Source: course PDF, pp. 8

Schematic: ESP32 Control Signals



CHANNEL A

PWMA → GPIO 18

AIN1/AIN2 → 25/27

CHANNEL B

PWMB → GPIO 19

BIN1/BIN2 → 26/33

ENABLE

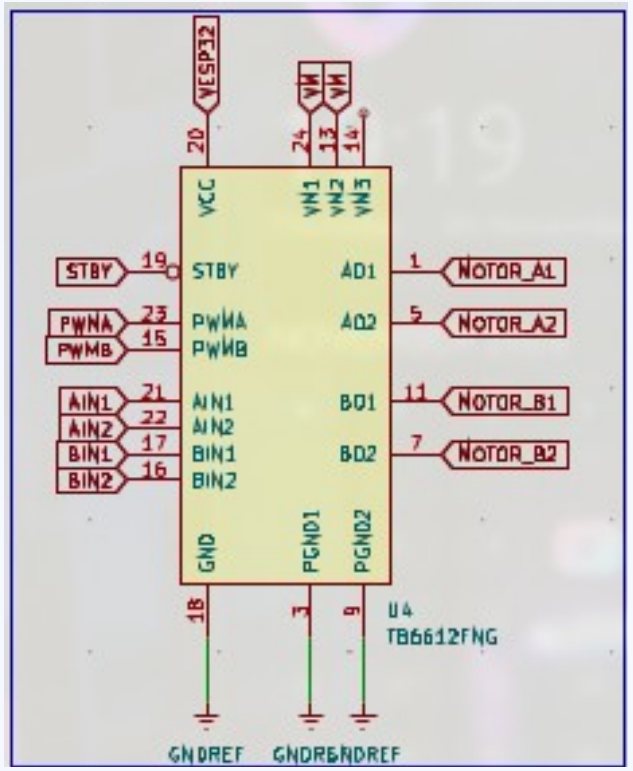
STBY → GPIO 5

Common GND reference

GPIO numbers identify ESP32 signals; they are not the driver IC's package pin numbers.

Source: course PDF, pp. 8

Schematic: TB6612FNG Connections



CONTROL

Direction inputs, PWM, and active-high STBY.

SUPPLIES

VCC is logic supply;
VM is motor supply.

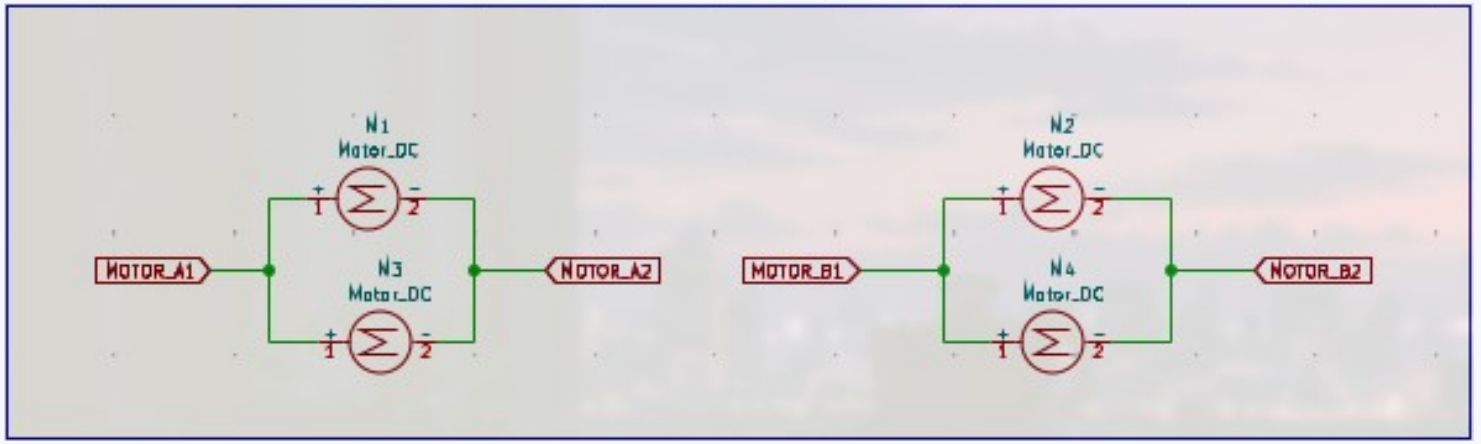
OUTPUTS

AO1/AO2 → left pair;
BO1/BO2 → right pair.

Connect power ground and logic ground to the common reference.

Source: course PDF, pp. 8

Schematic: Two Motors per Channel



LEFT PAIR

M1 and M3 share the channel A outputs.

RIGHT PAIR

M2 and M4 share the channel B outputs.

CHECK POLARITY

Paired wheels must propel the same side.

Parallel motors share voltage; their currents add at the driver channel.

Source: course PDF, pp. 8

Resolve the Supply Labels Before Wiring

Original drawing	The buck symbols are labeled 3.3 V and 12 V; the text describes 5 V VIN and a 6 V motor rail.
Logic supply	Use a compatible logic supply for 3.3 V ESP32 outputs; distinguish VCC from VM.
Motor rating	The notes list 5 V gearmotors. Confirm their allowed voltage before using the 6 V example.

Set the rail to suit the actual motor; lower duty still applies full-voltage PWM pulses.

Source: course PDF, pp. 8, 16

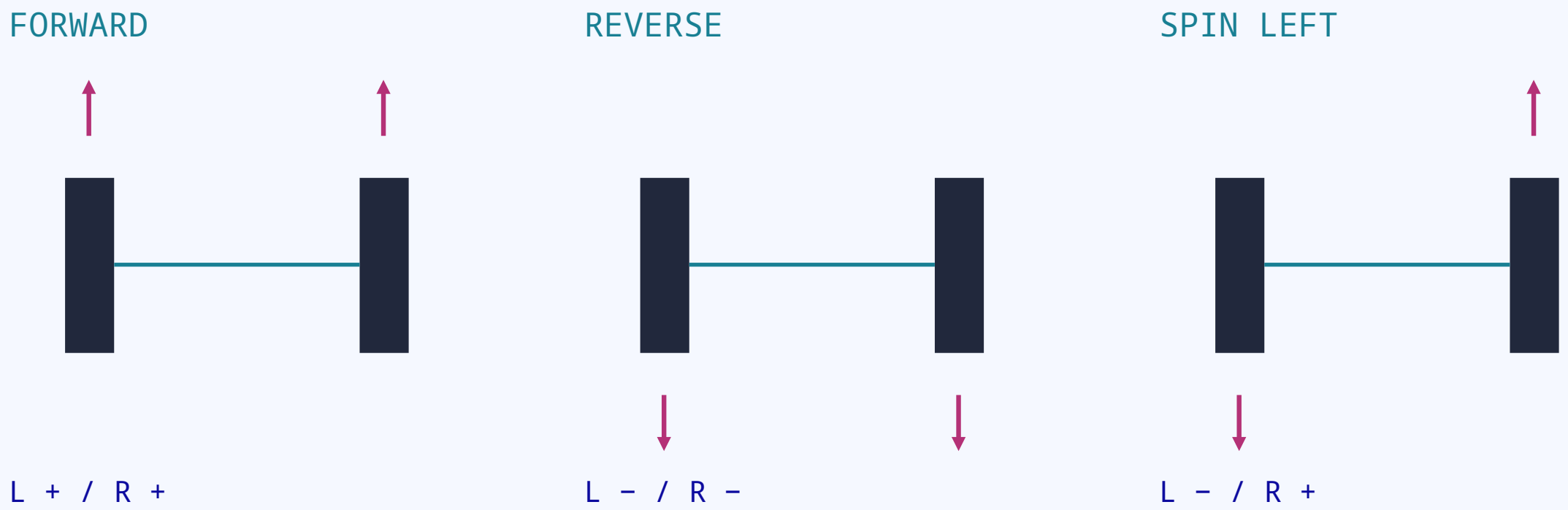
Firmware Pin Map

Function	Left / Channel A	Right / Channel B
PWM	GPIO 18 / LEDC 0	GPIO 19 / LEDC 1
Direction IN1	GPIO 25	GPIO 26
Direction IN2	GPIO 27	GPIO 33
Shared enable	STBY: GPIO 5	STBY: GPIO 5
Supporting I/O	Heartbeat: GPIO 4	Battery ADC: GPIO 32

The firmware uses two independent PWM channels at 20 kHz and 8-bit resolution.

Source: course PDF, pp. 8–10

Signed Commands Produce Chassis Motion



Check the physical wheel direction before interpreting command signs.

Source: course PDF, pp. 12–13

Calculate Duty and Expected Average Voltage

$$D = \frac{180}{255} \approx 0.7059 = 70.59\%$$

$$V_{\text{avg}} \approx 0.7059 \times 6\,\text{V} \approx 4.24\,\text{V}$$

$$N = -180 \quad \Rightarrow \quad V_{\text{avg}} \approx -4.24\,\text{V}$$

Ideal 6 V reference example; driver losses and meter response change measured values.

Source: course PDF, pp. 2, 13, 16

The Source Demo: An Eight-Second Sequence

Time in cycle	Left duty	Right duty	Motion
0–2 s	+180	+180	Forward
2–4 s	−180	−180	Reverse
4–6 s	−150	+150	Spin left
6–8 s	0	0	Stop

This is a step sequence, not a ramp. Test one mode at a time before sequencing.

Source: course PDF, pp. 13

Keep Motion and Logging at Different Rates

MOTION

100 Hz

Apply left/right commands every 10 ms.

TELEMETRY

10 Hz

Report duty, battery, and driver state.

HEARTBEAT

1 Hz task

Toggle the status LED every second.

Hardware PWM runs independently at 20 kHz.

Source: course PDF, pp. 13–16

Start with the Driver Disabled

```
pinMode(PIN_STBY, OUTPUT);
digitalWrite(PIN_STBY, LOW);

digitalWrite(PIN_AIN1, LOW);
digitalWrite(PIN_AIN2, LOW);
digitalWrite(PIN_BIN1, LOW);
digitalWrite(PIN_BIN2, LOW);
```

Configure direction pins as outputs first; keep STBY low during initialization.

Source: course PDF, pp. 10

Attach the Two PWM Channels

```
bool left_ok = ledcAttachChannel(
  PIN_PWMA, 20000, 8, LEDC_CHANNEL_LEFT);
bool right_ok = ledcAttachChannel(
  PIN_PWMB, 20000, 8, LEDC_CHANNEL_RIGHT);
if (!left_ok || !right_ok) return false;

ledcWrite(PIN_PWMA, 0);
ledcWrite(PIN_PWMB, 0);
```

Enable STBY only after PWM initialization succeeds and both duties are zero.

Source: course PDF, pp. 10–11

Positive Command: Forward Polarity

```
duty_count = constrain(duty_count, -255, 255);

if (duty_count > 0) {
  digitalWrite(PIN_AIN1, HIGH);
  digitalWrite(PIN_AIN2, LOW);
  ledcWrite(PIN_PWMA, uint8_t(duty_count));
}
```

Inside setLeftMotorSpeed(): magnitude controls duty; direction inputs set polarity.

Source: course PDF, pp. 11

Negative Command: Reverse Polarity

```
else if (duty_count < 0) {
  digitalWrite(PIN_AIN1, LOW);
  digitalWrite(PIN_AIN2, HIGH);

  ledcWrite(PIN_PWMA, uint8_t(-duty_count));
}
```

Mirror the same logic for channel B; manage reversal transients before high-load tests.

Source: course PDF, pp. 11–12

Zero Command: Explicit Short Brake

```
else {
  digitalWrite(PIN_AIN1, HIGH);
  digitalWrite(PIN_AIN2, HIGH);
  ledcWrite(PIN_PWMA, 0);
}
```

TB6612FNG: both direction inputs HIGH explicitly select short brake.

Source: course PDF, pp. 11–12

Apply Independent Left and Right Commands

```
void driveRobot(int16_t left, int16_t right) {
  left_command_duty = constrain(left, -255, 255);
  right_command_duty = constrain(right, -255, 255);

  setLeftMotorSpeed(left_command_duty);
  setRightMotorSpeed(right_command_duty);
}
```

Store the bounded commands so telemetry agrees with the PWM values sent to the driver.

Source: course PDF, pp. 12

Run the Motion Task at 100 Hz

```
if (uint32_t(current_us - last_motion_time_us)
    >= INTERVAL_MOTION_US) {

  last_motion_time_us += INTERVAL_MOTION_US;
  task_motion_control();
}
```

Use the existing 10 Hz telemetry and 1 Hz heartbeat checks alongside this one.

Source: course PDF, pp. 15–16

Bench Setup Before the First Motion

Power disconnected	Verify common ground, wiring polarity, and that no supply rail has a persistent short.
Wheels clear	Secure the chassis with wheels off the bench. Keep access to the power disconnect.
One step at a time	Start at low duty on one channel; confirm both paired wheels move in the intended direction.

Stop before reversing during initial tests; build up to the source demonstration.

Source: course PDF, pp. 8, 16

Verify Logic Supply, Motor Supply, and Enable

Logic rail	The course target is 5.00 V ± 0.05 V at ESP32 VIN; check driver VCC separately.
Motor rail	Measure VM relative to power ground. Use the voltage appropriate to the actual motors.
Standby input	STBY low disables the outputs. Confirm the intended enable level at the driver.

Check rail voltage under load, not only with the motors disconnected.

Source: course PDF, pp. 16

Measure Across the Motor Outputs

Probe location	Measure between OUT1 and OUT2, not only from either output to ground.
Polarity	The meter should change sign when the command direction is reversed.
Magnitude	At duty 180 and a 6 V reference rail, the ideal average is about ±4.24 V.

PWM measurements depend on the meter; a scope shows the actual switching waveform.

Source: course PDF, pp. 16

Debugging: No Motor Motion

Start with STBY	Check the enable pin at the driver, then check firmware initialization.
Check VM + GND	Verify supply, fuse, terminal connections, and ground continuity.
Then check inputs	Verify direction levels and PWM at the IC. Check output wiring and thermal state next.

A zero output alone does not prove the driver is damaged.

Source: course PDF, pp. 16–17

Debugging: Wrong Direction or Uneven Motion

One wheel fights	Check polarity of the two motors wired in parallel on that channel.
Chassis curves	Equal duty does not guarantee equal speed; load, motor tolerances, and friction can differ.
Reset or sag	Inspect the power and return paths; observe supply voltage during motor startup.

Separate wiring problems from the inherent limits of open-loop speed control.

Source: course PDF, pp. 1–2, 8, 16

Before Moving to Closed-Loop Control

Electrical behavior	Both channels respond to signed duty and have a verified stop/standby state.
Physical behavior	Wheel polarity and chassis motion agree with the intended command convention.
Next measurement	Encoders will measure actual motion so the controller can correct speed error.

Command → driver → motor is now the foundation for feedback control.

Source: course PDF, pp. 1–2, 8, 16–17

References

01 Espressif Systems — LED Control (LEDC)
Arduino-ESP32 API documentation

02 Texas Instruments — Motor Drivers Overview

03 Making Embedded Systems: Design Patterns for Great Software
Elecia White

04 Modern Robotics: Mechanics, Planning, and Control
Kevin M. Lynch and Frank C. Park

Source: course PDF, pp. 17

Thank you!

Questions & discussion

